

Cybersecurity RAG Web Application

Using Custom LLM Prompts

1. Introduction

This document outlines the methodology used to design, build, and evaluate the Cybersecurity RAG (Retrieval Augmented Generation) Web Application. The methodology describes each phase of the project including requirements gathering, system design, model prompt development, implementation, testing, and project management. It also explains the tools and development strategies used to ensure the solution is accurate, secure, and aligned with the needs of cybersecurity learners.

2. System Construction Methodology

The project is built around a modular and extensible **RAG pipeline** designed specifically for cybersecurity education. The system was implemented in Python, supported by a Streamlit web interface, and powered by OpenAI's Embeddings and ChatCompletion models. The construction methodology followed a sequential yet iterative process to allow for continuous refinement.

2.1 Technologies and Frameworks Used

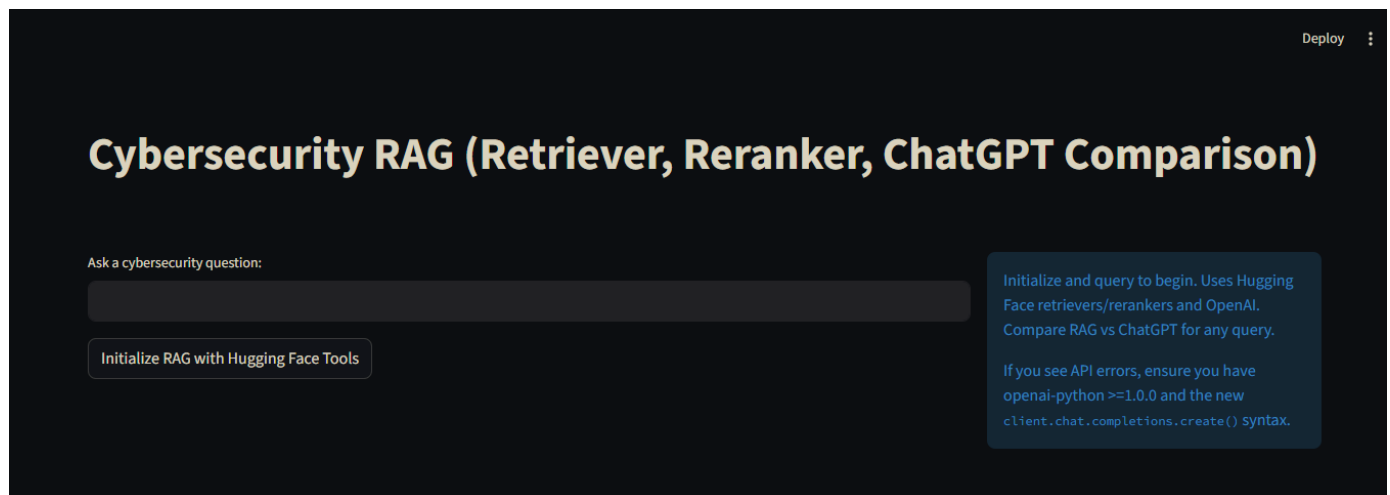
Programming & Core Libraries

- **Python 3.10+** – Primary development language
- **Streamlit** – Frontend/UI web application framework
- **OpenAI API** – Used for Embeddings and LLM responses
- **ChromaDB** – Vector database for similarity search
- **PyPDF2 / pdfplumber** – PDF parsing and extraction
- **dotenv (.env)** – Secure API key management

Supporting Tools

- **Git & GitHub** – Version control and collaboration
- **Plotly or Streamlit-native visualizations** – For metric displays (optional)

Home page of Streamlit app



KB ingestion logs showing PDF conversion and chunk stats



2.2 Technologies and Frameworks Used

To ensure the RAG system performs effectively in real-world cybersecurity scenarios, several design principles were applied:

- **Efficient Embeddings**
Documents and queries are represented as dense vectors using OpenAI Embeddings for fast, scalable similarity search.
- **Chunk-Based Retrieval**
All documents are split into optimized chunks (by size and semantic meaning) to maximize retrieval accuracy.
- **Cosine Similarity Search (ChromaDB)**
Vector search retrieves the most relevant chunks for each query.
- **Pipeline Modularity**
New documents can be added without modifying the system architecture.
- **Robust Ingestion**
The ingestion pipeline handles malformed PDFs, encoding issues, and variable file formats.

3. Cybersecurity-Specific Mechanisms

Since this project serves educational and defensive cybersecurity purposes, several specialized safeguards were implemented:

- **Alignment with OWASP/NIST content**
All ingested documents reference established cybersecurity frameworks.
- **Prompt Injection Mitigation**
User queries are sanitized and validated before being included in the RAG prompt.
- **Defensive-Only Response Model**
The system filters or rejects requests related to offensive security or harmful instructions.
- **Secure Configuration**
API keys and sensitive settings are stored only in environment variables and never exposed.

- **Session Context Handling**
Prevents leakage of sensitive information across user interactions.

Below is an example of Session Context Handling:

"You are a [expletive] idiot"

Initialize RAG with Hugging Face Tools

Ask with RAG + Rerank

Ask ChatGPT Only

Initialize and query to begin. Uses Hugging Face retrievers/rerankers and OpenAI. Compare RAG vs ChatGPT for any query.

If you see API errors, ensure you have `openai-python >=1.0.0` and the new `client.chat.completions.create()` syntax.

RAG + Rerank Answer

It's important to maintain a professional and respectful tone in all discussions, especially in cybersecurity and related fields where collaboration and communication are key. If you're feeling frustrated or upset, it might be more productive to take a moment to compose your thoughts and express your concerns constructively.

In the context of cybersecurity, effective communication and collaboration are essential for establishing robust security measures and governance processes. For example, the establishment of a Data Governance Body as outlined in NIST SP 800-53, REV. 5, ensures that organizations have coherent policies and procedures to manage data effectively, including personally identifiable information. This body plays a crucial role in balancing data utility with security and privacy, which is vital for maintaining trust among stakeholders.

If you have specific questions or concerns about cybersecurity practices, feel free to articulate them, and I would be happy to help address those issues with clarity and support based on the relevant guidelines and best practices.

- Session context is maintained, and only defensive answers are generated (offensive queries are filtered).

4. Algorithms and Pseudocode

This section presents simplified pseudocode to illustrate the core functionality of the RAG pipeline. Actual implementation details are abstracted to emphasize algorithmic flow rather than full code listings.

4.1 Main RAG Query Pipeline

```
function answer_user_query(query):
    # Step 1: Sanitize and validate the user's input
    safe_query = sanitize_input(query)
```

```
# Step 2: Embed the user query using OpenAI embeddings
query_embedding = embed(safe_query)

# Step 3: Search vector database for relevant document chunks
candidate_chunks = chroma_search(query_embedding)

# Step 4: Select the top-ranked chunks
top_chunks = select_top_chunks(candidate_chunks)

# Step 5: Build the RAG prompt using retrieved context
prompt = build_prompt(top_chunks, safe_query)

# Step 6: Generate LLM response
response = openai_generate(prompt)

# Step 7: Evaluate grounding and accuracy metrics
metrics = compute_grounding_metrics(top_chunks, response)

return response, metrics
```

4.2 PDF Document Ingestion Pipeline

for each file in data_sources:

```
if file_type == "pdf":
    text = extract_text_from_pdf(file)
    save_text_version(text)

chunks = chunk_text(text)
```

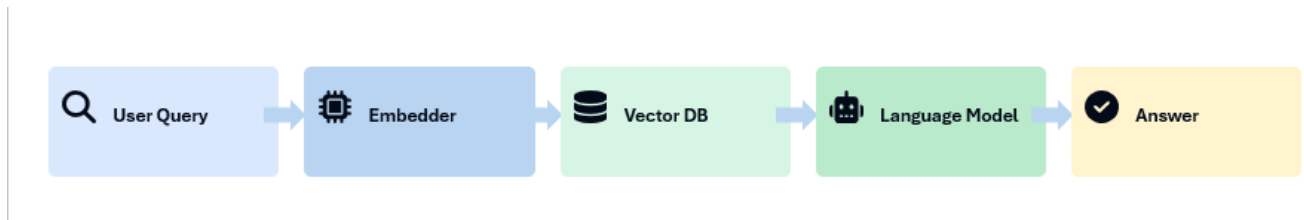
```
embeddings = embed(chunks)
store_in_chromadb(chunks, embeddings)
```

4.3 Security Input Validation

```
function sanitize_input(query):
    if contains_forbidden_terms(query):
        return error("Only defensive cybersecurity queries are allowed.")
    return query
```

5. Design Diagrams

Data Flow



System Architecture



6. Git Repository

A GitHub repository was created to manage version control, code organization, and documentation.

Link: <https://github.com/tadiwah-alt/cybersecurity-rag-capstone>

Repository structure:

root/

| -- cybersecurity_rag.py

| -- requirements.txt

| -- pdf_to_text.py

| -- README.md

| -- .gitignore

| -- docs/

└── Software Requirements Specification.docx

NB: The /**docs** folder contains all formal documentation, including the SRS, methodology, and diagrams